

Minimum Spanning Trees

Comp Sci 214, Spring 2025

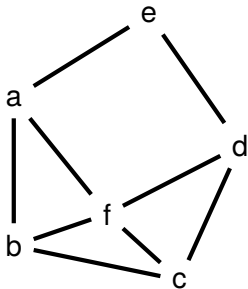
Don't forget to check in on Youhere!

Announcement

- Disjoint sets worksheet
 - ▶ Extra practice; not part of final grade
- Will have two more like that next week
 - ▶ AVL trees
 - ▶ Relational model

Definition: spanning tree

A *spanning tree* is a cycle-free subset of edges (i.e., a tree) that connects every vertex in a connected component:

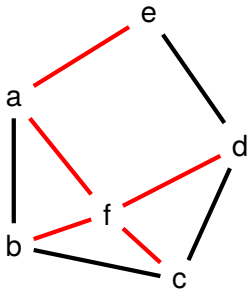


Applications:

- Paths for broadcasts on a computer network:
computers = nodes, cables = edges, weights = latency
- Highest priority streets to plow in a snowstorm:
intersections = nodes, streets = edges, weight = distance
- Etc.

Definition: spanning tree

A *spanning tree* is a cycle-free subset of edges (i.e., a tree) that connects every vertex in a connected component:

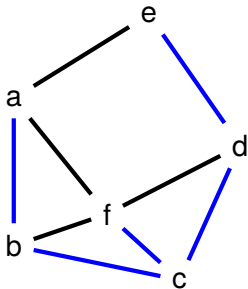


Applications:

- Paths for broadcasts on a computer network:
computers = nodes, cables = edges, weights = latency
- Highest priority streets to plow in a snowstorm:
intersections = nodes, streets = edges, weight = distance
- Etc.

Definition: spanning tree

A *spanning tree* is a cycle-free subset of edges (i.e., a tree) that connects every vertex in a connected component:

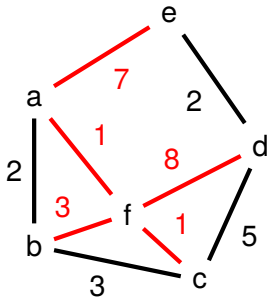
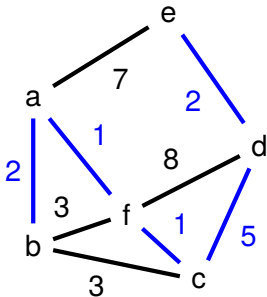


Applications:

- Paths for broadcasts on a computer network:
computers = nodes, cables = edges, weights = latency
- Highest priority streets to plow in a snowstorm:
intersections = nodes, streets = edges, weight = distance
- Etc.

Definition: *minimum* spanning tree (MST)

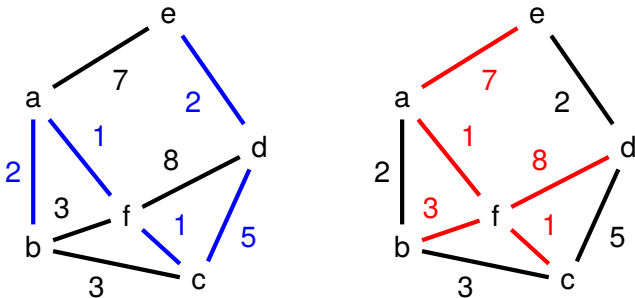
In a weighted graph, a spanning tree is *minimal* if the sum of its weights is minimal over all possible spanning trees:



Left tree is minimal; right is not.

Definition: *minimum* spanning tree (MST)

In a weighted graph, a spanning tree is *minimal* if the sum of its weights is minimal over all possible spanning trees:

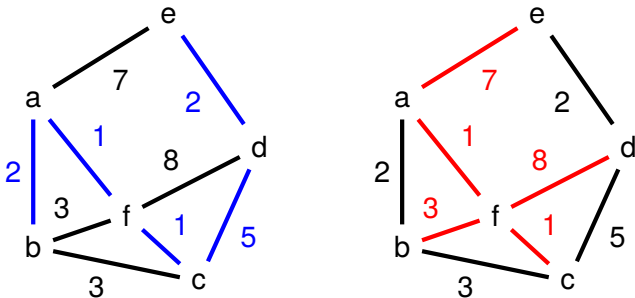


Fun fact: the MST problem was formulated in 1926 to design the electrical grid of Moravia (eastern Czech Republic) using the least amount of cables possible.

cities = nodes, power lines = edges, weights = distance

Definition: *minimum* spanning tree (MST)

In a weighted graph, a spanning tree is *minimal* if the sum of its weights is minimal over all possible spanning trees:



Note: paths between nodes in an MST may not be the shortest!
No guarantee of that; only about total sum of weights.
In general, can't do MST and SSSP with the same tree!
Prof. Khuller designed an algorithm that's a good compromise:
within constant factors of optimal (SODA 1993).

Computing an MST

It's surprisingly easy—there are two simple, greedy algorithms:

- Prim's
- Kruskal's

Greedy algorithm: at each step, pick the choice that looks best *right now*. No need to think ahead or backtrack.

Computing an MST

It's surprisingly easy—there are two simple, greedy algorithms:

- Prim's
- Kruskal's

Greedy algorithm: at each step, pick the choice that looks best *right now*. No need to think ahead or backtrack.

These two illustrate a nice tradeoff!

- Same complexity, but...
 - ▶ **Prim:** simple data, complex code!
 - ▶ **Kruskal:** slightly more complex data, very simple code!
- Smart data, stupid code (alt., simple) is usually better
 - ▶ Complex bits are tucked away in a data structure!
 - ▶ **Today:** Kruskal's algorithm, and the smart data it needs

Stop! Question time!

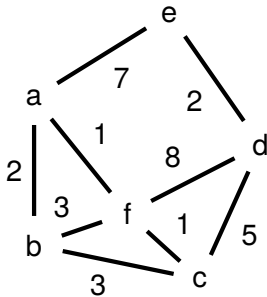
Before we look at Kruskal's algorithm...

Anything unclear so far?

Anything I missed?

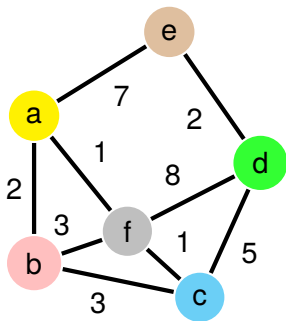
Kruskal's algorithm

1. Start with a trivial tree at every vertex
2. Consider the edges in order from smallest to largest
 - Can either use a priority queue or some other way of sorting
3. If an edge would join two separate trees, add the edge and merge the two small trees into a larger one



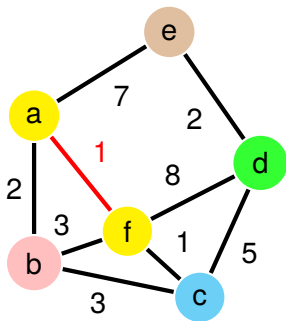
Kruskal's algorithm

1. Start with a trivial tree at every vertex
2. Consider the edges in order from smallest to largest
 - Can either use a priority queue or some other way of sorting
3. If an edge would join two separate trees, add the edge and merge the two small trees into a larger one



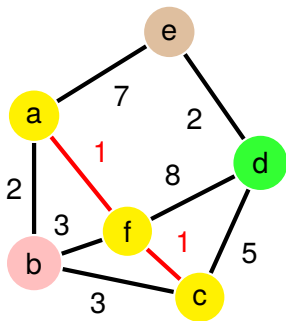
Kruskal's algorithm

1. Start with a trivial tree at every vertex
2. Consider the edges in order from smallest to largest
 - Can either use a priority queue or some other way of sorting
3. If an edge would join two separate trees, add the edge and merge the two small trees into a larger one



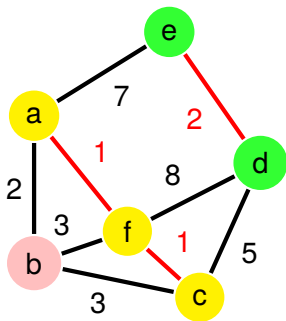
Kruskal's algorithm

1. Start with a trivial tree at every vertex
2. Consider the edges in order from smallest to largest
 - Can either use a priority queue or some other way of sorting
3. If an edge would join two separate trees, add the edge and merge the two small trees into a larger one



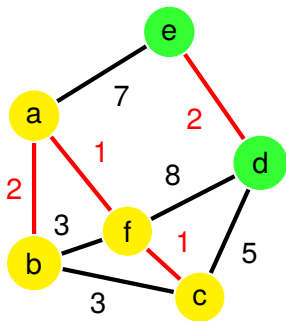
Kruskal's algorithm

1. Start with a trivial tree at every vertex
2. Consider the edges in order from smallest to largest
 - Can either use a priority queue or some other way of sorting
3. If an edge would join two separate trees, add the edge and merge the two small trees into a larger one



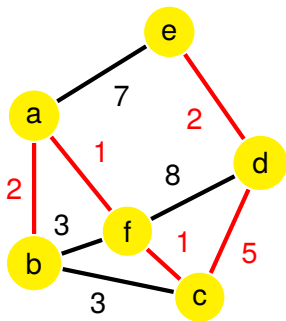
Kruskal's algorithm

1. Start with a trivial tree at every vertex
2. Consider the edges in order from smallest to largest
 - Can either use a priority queue or some other way of sorting
3. If an edge would join two separate trees, add the edge and merge the two small trees into a larger one



Kruskal's algorithm

1. Start with a trivial tree at every vertex
2. Consider the edges in order from smallest to largest
 - Can either use a priority queue or some other way of sorting
3. If an edge would join two separate trees, add the edge and merge the two small trees into a larger one



Bonus: finds min spanning *forests* for disconnected graphs!

Kruskal's algorithm

Let's use sets to represent colors!

- Initially, all different (singleton) sets / colors
- Union sets / merge colors as we go
- All the sets are *disjoint*: no overlap, and no orphans

Input: A weighted graph g of n vertices

$forest \leftarrow$ a graph of n vertices and 0 edges;

```
for each edge  $(u, v)$  in increasing order of weight  $w$  do
    if not in_same_set?( $u, v$ ) then
        union(set of  $u$ , set of  $v$ );
        forest.add_edge( $u, v$ );
    end
end
```

Very simple! (And much simpler than Prim!)

Kruskal's algorithm

Let's use sets to represent colors!

- Initially, all different (singleton) sets / colors
- Union sets / merge colors as we go
- All the sets are *disjoint*: no overlap, and no orphans

Input: A weighted graph g of n vertices

$forest \leftarrow$ a graph of n vertices and 0 edges;

```
for each edge  $(u, v)$  in increasing order of weight  $w$  do
    if not in_same_set?  $(u, v)$  then
        union(set of  $u$ , set of  $v$ );
         $forest.add\_edge(u, v)$ ;
    end
end
```

Very simple! (And much simpler than Prim!)

But we've skipped some details about the sets...

Kruskal's algorithm

Main challenge: keeping track of the disjoint sets (colors)

- So we know which vertices are part of the same set (in_same_set?)
- And we know which two sets get combined when adding an edge to the tree (union)

We **could** use general-purpose sets for that...

Kruskal's algorithm

Main challenge: keeping track of the disjoint sets (colors)

- So we know which vertices are part of the same set (in_same_set?)
- And we know which two sets get combined when adding an edge to the tree (union)

We **could** use general-purpose sets for that...

Or we could introduce a new ADT that specializes in *just* these two operations, specifically on sets that we know are disjoint!

- → can potentially make them more efficient (smart data)
- This is what we'll do

Disjoint Sets ADT (aka Union-Find)

Universe: set containing all the things we're interested in.
(e.g., all the nodes in our graph)

Want to *partition* the universe: organize elements into (sub)sets that do not overlap (*disjoint*). (e.g., colors for the partial trees)

Each set has one, distinguished representative object.

$\{\underline{0}\} \{1, \underline{2}, 5\} \{\underline{3}, 7\} \{\underline{4}\} \{\underline{6}\}$

```
interface DISJOINT_SETS:
  def len(self) -> nat?
  def union(self, p: nat?, q: nat?) -> NoneC
  def find(self, p: nat?) -> nat?
```

Behavior:

- `d.union(p, q)` joins `p` and `q`'s sets
- `d.find(p)` returns `p`'s representative (i.e., which set it's in)
- `in_same_set?(p, q) → d.find(p) == d.find(q)`

Disjoint Sets Example

$$d = \boxed{\{\underline{0}\} \{\underline{1}\} \{\underline{2}\} \{\underline{3}\} \{\underline{4}\} \{\underline{5}\} \{\underline{6}\} \{\underline{7}\}}$$

Disjoint Sets Example

$d = \boxed{\{\underline{0}\} \{\underline{1}\} \{\underline{2}\} \{\underline{3}\} \{\underline{4}\} \{\underline{5}\} \{\underline{6}\} \{\underline{7}\}}$

`d.find(0) ⇒ ???`

Disjoint Sets Example

$d = \boxed{\{\underline{0}\} \{\underline{1}\} \{\underline{2}\} \{\underline{3}\} \{\underline{4}\} \{\underline{5}\} \{\underline{6}\} \{\underline{7}\}}$

$d.\text{find}(0) \Rightarrow 0$

Disjoint Sets Example

$d = \boxed{\{\underline{0}\} \{\underline{1}\} \{\underline{2}\} \{\underline{3}\} \{\underline{4}\} \{\underline{5}\} \{\underline{6}\} \{\underline{7}\}}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow ???$

Disjoint Sets Example

$d = \boxed{\{\underline{0}\} \{\underline{1}\} \{\underline{2}\} \{\underline{3}\} \{\underline{4}\} \{\underline{5}\} \{\underline{6}\} \{\underline{7}\}}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

Disjoint Sets Example

$d = \boxed{\{\underline{0}\} \{\underline{1}\} \{\underline{2}\} \{\underline{3}\} \{\underline{4}\} \{\underline{5}\} \{\underline{6}\} \{\underline{7}\}}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

Disjoint Sets Example

$d = \boxed{\{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

$d = \boxed{\{0\} \{1\} \{2, 5\} \{3\} \{4\} \{6\} \{7\}}$

For this example, we are picking reps arbitrarily.

Actual data structures will have rules for picking.

$d.find(2) \Rightarrow ???$

Disjoint Sets Example

$d = \boxed{\{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

$d = \boxed{\{0\} \{1\} \{2, 5\} \{3\} \{4\} \{6\} \{7\}}$

For this example, we are picking reps arbitrarily.

Actual data structures will have rules for picking.

$d.find(2) \Rightarrow 2$

Disjoint Sets Example

$d = \boxed{\{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

$d = \boxed{\{0\} \{1\} \{2, 5\} \{3\} \{4\} \{6\} \{7\}}$

For this example, we are picking reps arbitrarily.

Actual data structures will have rules for picking.

$d.find(2) \Rightarrow 2$ $d.find(5) \Rightarrow ???$

Disjoint Sets Example

$d = \boxed{\{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

$d = \boxed{\{0\} \{1\} \{2, 5\} \{3\} \{4\} \{6\} \{7\}}$

For this example, we are picking reps arbitrarily.

Actual data structures will have rules for picking.

$d.find(2) \Rightarrow 2$ $d.find(5) \Rightarrow 2$

Disjoint Sets Example

$d = \boxed{\{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

$d = \boxed{\{0\} \{1\} \{2, 5\} \{3\} \{4\} \{6\} \{7\}}$

For this example, we are picking reps arbitrarily.

Actual data structures will have rules for picking.

$d.find(2) \Rightarrow 2$ $d.find(5) \Rightarrow 2$

$d.union(1, 2)$

Disjoint Sets Example

$d = \{\{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}\}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

$d = \{\{0\} \{1\} \{2, 5\} \{3\} \{4\} \{6\} \{7\}\}$

For this example, we are picking reps arbitrarily.

Actual data structures will have rules for picking.

$d.find(2) \Rightarrow 2$ $d.find(5) \Rightarrow 2$

$d.union(1, 2)$

$d = \{\{0\} \{1, 2, 5\} \{3\} \{4\} \{6\} \{7\}\}$

Disjoint Sets Example

$d = \{\{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}\}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

$d = \{\{0\} \{1\} \{2, 5\} \{3\} \{4\} \{6\} \{7\}\}$

For this example, we are picking reps arbitrarily.

Actual data structures will have rules for picking.

$d.find(2) \Rightarrow 2$ $d.find(5) \Rightarrow 2$

$d.union(1, 2)$

$d = \{\{0\} \{1, 2, 5\} \{3\} \{4\} \{6\} \{7\}\}$

$d.union(3, 7)$

Disjoint Sets Example

$d = \{\{0\} \{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}\}$

$d.find(0) \Rightarrow 0$ $d.find(1) \Rightarrow 1$

$d.union(2, 5)$

$d = \{\{0\} \{1\} \{2, 5\} \{3\} \{4\} \{6\} \{7\}\}$

For this example, we are picking reps arbitrarily.

Actual data structures will have rules for picking.

$d.find(2) \Rightarrow 2$ $d.find(5) \Rightarrow 2$

$d.union(1, 2)$

$d = \{\{0\} \{1, 2, 5\} \{3\} \{4\} \{6\} \{7\}\}$

$d.union(3, 7)$

$d = \{\{0\} \{1, 2, 5\} \{3, 7\} \{4\} \{6\}\}$

Kruskal's algorithm using disjoint sets

Input: A weighted *graph* of n vertices

Output: A minimum spanning *forest* (represented as a graph)

$ds \leftarrow$ a new disjoint sets universe with n objects;

$forest \leftarrow$ a graph of n vertices and 0 edges;

```
for each edge  $(u, v)$  in increasing order of weight  $w$  do
    if  $ds.find(u) \neq ds.find(v)$  then
         $ds.union(u, v)$ ;
         $forest.add\_edge(u, v)$ 
    end
end
```

$|E| \log |E|$ to sort edges / initialize priority queue.

Then the loop runs for $|E|$ iterations.

Each iteration does at most 2 finds and 1 union.

Can do add_edge in $\mathcal{O}(1)$ (with an adjacency matrix).

So algorithm is $\mathcal{O}\left(|E| \log |E| + |E|(2 \times T_{find} + T_{union} + \mathcal{O}(1))\right)$

Stop! Question time!

Before we move on to disjoint sets implementations...

Anything unclear so far?

Anything I missed?

Implementing Disjoint Sets

First Attempt: Quick-Find

Array where array elt i holds representative for universe elt i .

Example:

{ <u>0</u> }	{ <u>1</u> }	{2, 3, 4, <u>9</u> }	{5, <u>6</u> }	{ <u>7</u> }	{ <u>8</u> }
--------------	--------------	----------------------	----------------	--------------	--------------

	0	1	2	3	4	5	6	7	8	9
id:	0	1	9	9	9	6	6	7	8	9

- **Find:** return representative ($\text{id}[i]$)
- **Union** p and q : change all entries with p 's id to q 's id
 - ▶ Always same direction (code needs a uniform rule!)

First Attempt: Quick-Find

Array where array elt i holds representative for universe elt i .

Example:

{ <u>0</u> }	{ <u>1</u> }	{2, 3, 4, <u>9</u> }	{5, <u>6</u> }	{ <u>7</u> }	{ <u>8</u> }
--------------	--------------	----------------------	----------------	--------------	--------------

	0	1	2	3	4	5	6	7	8	9
id:	0	1	9	9	9	6	6	7	8	9

- **Find:** return representative ($\text{id}[i]$)
- **Union** p and q : change all entries with p 's id to q 's id
 - ▶ Always same direction (code needs a uniform rule!)

$\text{d.union}(3, 5) \rightarrow$

{ <u>0</u> }	{ <u>1</u> }	{2, 3, 4, 5, <u>6</u> , 9}	{ <u>7</u> }	{ <u>8</u> }
--------------	--------------	----------------------------	--------------	--------------

	0	1	2	3	4	5	6	7	8	9
id:	0	1	6	6	6	6	6	7	8	6

Many elements may need to change!

Quick-Find is Too Slow

- **Find:** return representative (`id[i]`)
- **Union** p and q : change all entries with p 's id to q 's id
 - ▶ Always same direction (code needs a uniform rule!)

`find` is just array reference: $\mathcal{O}(1)$

`union` loops over the array to find what needs to change: $\mathcal{O}(n)$

Not good. Let's try something else.

Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

Note: This is just a *visualisation*, not a *representation*.

With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	3	4	5	6	7	8	9

0 1 2 3 4 5 6 7 8 9

Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

Note: This is just a *visualisation*, not a *representation*.

With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

`d.union(3, 4)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	3	4	5	6	7	8	9

0 1 2 3 4 5 6 7 8 9

Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

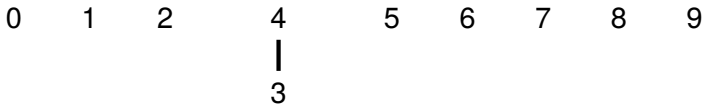
Note: This is just a *visualisation*, not a *representation*.

With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

`d.union(3, 4)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	4	4	5	6	7	8	9



Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

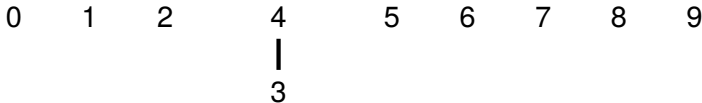
Note: This is just a *visualisation*, not a *representation*.

With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

`d.union(4, 9)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	4	4	5	6	7	8	9



Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

Note: This is just a *visualisation*, not a *representation*.

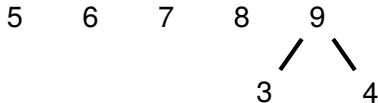
With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

`d.union(4, 9)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	9	9	5	6	7	8	9

0 1 2



Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

Note: This is just a *visualisation*, not a *representation*.

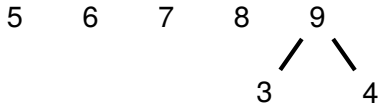
With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

`d.union(5, 6)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	9	9	5	6	7	8	9

0 1 2



Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

Note: This is just a *visualisation*, not a *representation*.

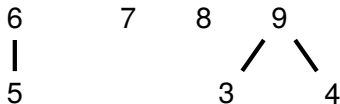
With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

`d.union(5, 6)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	9	9	6	6	7	8	9

0 1 2



Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

Note: This is just a *visualisation*, not a *representation*.

With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

`d.union(3, 5)`

0	1	2	3	4	5	6	7	8	9
0	1	2	9	9	6	6	7	8	9

id:

0 1 2

6

7

8

9

5

3

4

Visualizing Disjoint Sets as Forests

Can think of sets as trees, reps as roots, universe as forest.

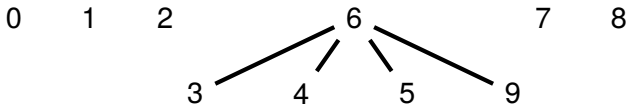
Note: This is just a *visualisation*, not a *representation*.

With Quick-Find:

- **Union** p and q : change all entries with p 's id to q 's id
= turning p and its children into children of q 's root.

`d.union(3, 5)`

0	1	2	3	4	5	6	7	8	9
0	1	2	6	6	6	6	7	8	6



Trees remain flat $\rightarrow$ find remains fast!

Need to update all a tree's nodes $\rightarrow$ union is slow...

Could we try some other way of merging trees?

Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(3, 4)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	3	4	5	6	7	8	9

0 1 2 3 4 5 6 7 8 9

Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

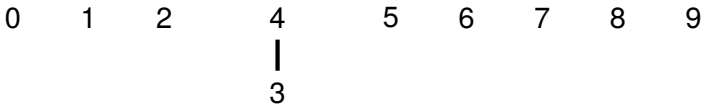
- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(3, 4)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	4	4	5	6	7	8	9



Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

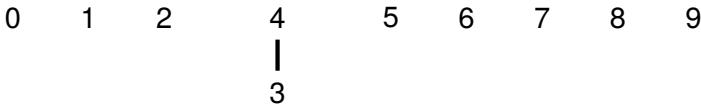
- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(2, 9)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	2	4	4	5	6	7	8	9



Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

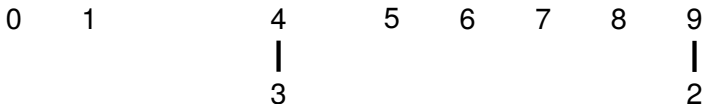
- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(2, 9)`

0	1	2	3	4	5	6	7	8	9
0	1	9	4	4	5	6	7	8	9



Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

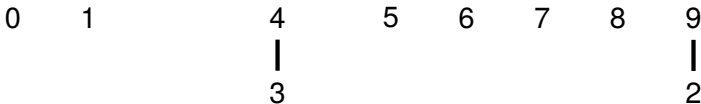
- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

```
d.union(5, 6)
```

	0	1	2	3	4	5	6	7	8	9
id:	0	1	9	4	4	5	6	7	8	9



Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(5, 6)`

0	1	2	3	4	5	6	7	8	9
0	1	9	4	4	6	6	7	8	9

id:



Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(3, 2)`

0	1	2	3	4	5	6	7	8	9
0	1	9	4	4	6	6	7	8	9



Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(3, 2)`

0	1	2	3	4	5	6	7	8	9
0	1	9	4	9	6	6	7	8	9

id:

0 1

6

7

8

9

5

2

4

3

Only one element ever needs to change!

Second Attempt: Quick-Union

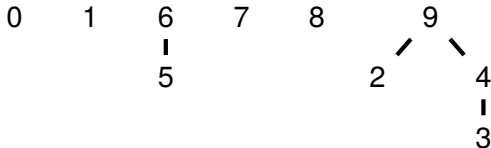
Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Find: Go until `id` doesn't change; root is its own representative.

`d.find(3)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	9	4	9	6	6	7	8	9



Second Attempt: Quick-Union

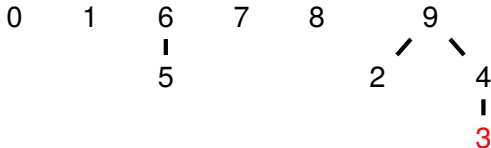
Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Find: Go until `id` doesn't change; root is its own representative.

`d.find(3)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	9	4	9	6	6	7	8	9



Second Attempt: Quick-Union

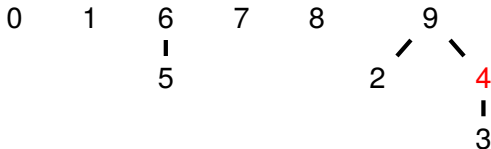
Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Find: Go until `id` doesn't change; root is its own representative.

`d.find(3)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	9	4	9	6	6	7	8	9



Second Attempt: Quick-Union

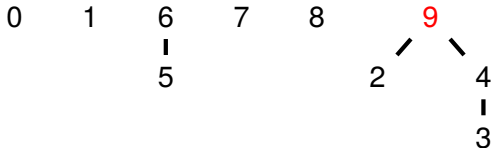
Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Find: Go until `id` doesn't change; root is its own representative.

`d.find(3)`

	0	1	2	3	4	5	6	7	8	9
id:	0	1	9	4	9	6	6	7	8	9



9 is its own root, we're done. 9 is 3's representative.

Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

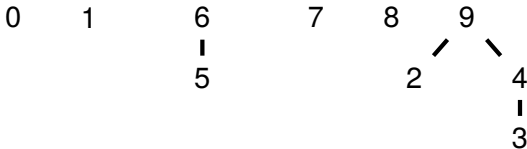
Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(3, 5)`

id:

0	1	9	4	9	6	6	7	8	9
---	---	---	---	---	---	---	---	---	---



Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(3, 5)`

id:

0	1	2	3	4	5	6	7	8	9
0	1	9	4	9	6	6	7	8	6



Second Attempt: Quick-Union

Array where array elt i holds *parent* for universe elt i .

- Reverse tree, as we saw for path trees
- Representative is `id[id[id[... id[i] ...]]]`

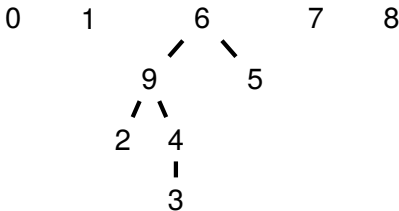
Union p and q : change id of p 's root to be q 's root

- Again, always in the same direction

`d.union(3, 5)`

id:

0	1	9	4	9	6	6	7	8	6
---	---	---	---	---	---	---	---	---	---



Trees can get tall $\rightarrow$ find can get slow (Cf. `d.find(3)`)

Quick-Union is also Too Slow

Quick-Find

- Trees are flat, but it's expensive to keep them flat
- find is $\mathcal{O}(1)$
- union is $\mathcal{O}(n)$

Quick-Union

- Trees can get tall
- So find can get as bad as $\mathcal{O}(n)$

Quick-Union is also Too Slow

Quick-Find

- Trees are flat, but it's expensive to keep them flat
- find is $\mathcal{O}(1)$
- union is $\mathcal{O}(n)$

Quick-Union

- Trees can get tall
- So find can get as bad as $\mathcal{O}(n)$
- *But* union needs to find roots, so also $\mathcal{O}(n)$! Oops.

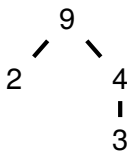
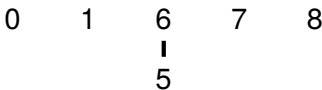
But still, we're on the right track!

Third Attempt: *Weighted* Quick-Union (WQU)

Idea: modify quick-union to avoid tall trees

id:

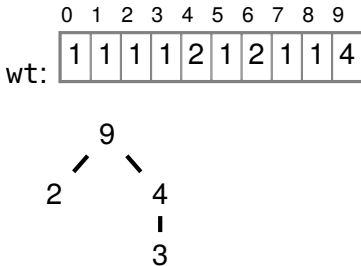
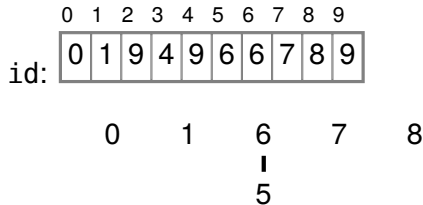
0	1	9	4	9	6	6	7	8	9
---	---	---	---	---	---	---	---	---	---



Third Attempt: *Weighted* Quick-Union (WQU)

Idea: modify quick-union to avoid tall trees

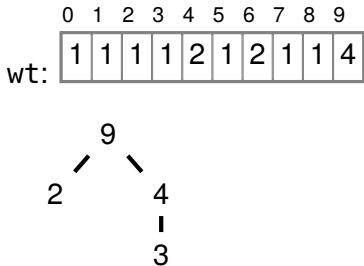
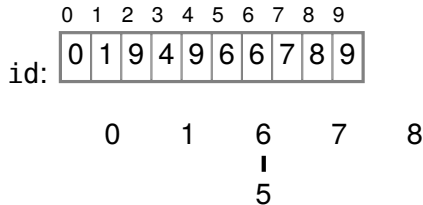
- Keep track of the size (# of nodes) of each tree
 - ▶ Keeping track of height works too, but more work



Third Attempt: *Weighted* Quick-Union (WQU)

Idea: modify quick-union to avoid tall trees

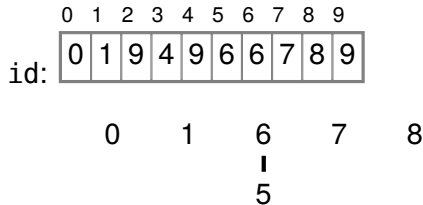
- Keep track of the size (# of nodes) of each tree
 - ▶ Keeping track of height works too, but more work
- Keep things balanced by linking smaller trees below larger



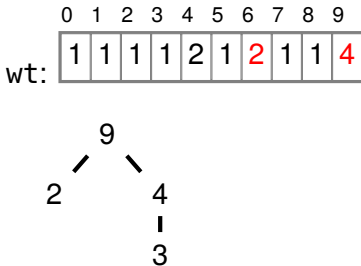
Third Attempt: *Weighted* Quick-Union (WQU)

Idea: modify quick-union to avoid tall trees

- Keep track of the size (# of nodes) of each tree
 - ▶ Keeping track of height works too, but more work
- Keep things balanced by linking smaller trees below larger



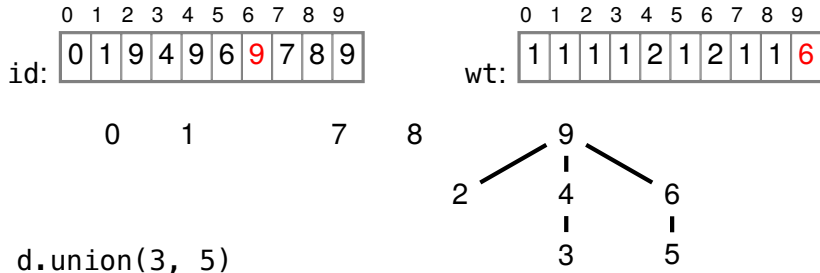
d.union(3, 5)



Third Attempt: *Weighted* Quick-Union (WQU)

Idea: modify quick-union to avoid tall trees

- Keep track of the size (# of nodes) of each tree
 - ▶ Keeping track of height works too, but more work
- Keep things balanced by linking smaller trees below larger



With quick-union: 9 under 6, height 4...

With *weighted* quick-union: 6 under 9, height 3!

WQU is Reasonably Fast

By linking smaller trees below larger ones, balance guaranteed.

- $\rightarrow$ height is $\mathcal{O}(\log n)$
- Proof is complicated. Let's take it on faith.

- **Find:** get to root, $\mathcal{O}(\log n)$
- **Union** p and q : find two roots, change one, $\mathcal{O}(\log n)$

WQU is Reasonably Fast

By linking smaller trees below larger ones, balance guaranteed.

- $\rightarrow$ height is $\mathcal{O}(\log n)$
- Proof is complicated. Let's take it on faith.

- **Find:** get to root, $\mathcal{O}(\log n)$
- **Union** p and q : find two roots, change one, $\mathcal{O}(\log n)$

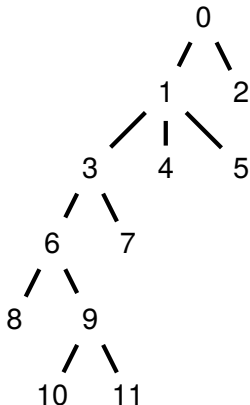
But can we do better?

Fourth Attempt: WQU with *Path Compression* (WQUPC)

Dear Lord, he's still going!

Idea: after we find the root of a tree, set the id of all the nodes we saw along the way to be the root.

`d.find(9)`

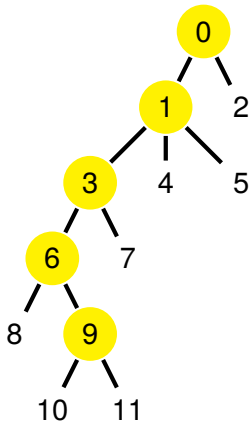


Fourth Attempt: WQU with *Path Compression* (WQUPC)

Dear Lord, he's still going!

Idea: after we find the root of a tree, set the id of all the nodes we saw along the way to be the root.

`d.find(9)`

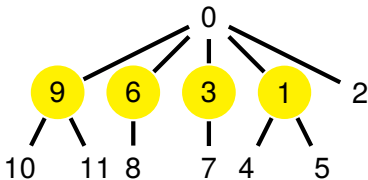


Fourth Attempt: WQU with *Path Compression* (WQUPC)

Dear Lord, he's still going!

Idea: after we find the root of a tree, set the id of all the nodes we saw along the way to be the root.

`d.find(9)`



Next time we find 3, 6, or 9, will be faster!

Can update parents as we return from recursive calls to find the root.
Easy, and no extra cost!

WQUPC is Crazy Fast

Trees stay *really* shallow.

m union and find operations on n objects is $\mathcal{O}(m \times \alpha(n))$

WQUPC is Crazy Fast

Trees stay *really* shallow.

m union and find operations on n objects is $\mathcal{O}(m \times \alpha(n))$

From Wikipedia:

$\alpha(n)$ is the inverse Ackermann function. This function has a value $\alpha(n) < 5$ for any value of n that can be written in this physical universe.

- AKA “I can’t believe it’s not constant!”
- $\mathcal{O}(m \times \alpha(n))$ is for m operations, not just one!
 - ▶ If you do enough operations, each one is almost constant!
 - ▶ Cf. amortized analysis lecture
- Proof is terrifying. I don’t think I’ve ever understood it.
 - ▶ But algorithm is still simple! (Used to be homework 6.)

Back to Kruskal

We said Kruskal is: $\mathcal{O}(|E| \log |E| + |E|(2 \times T_{find} + T_{union}))$

Back to Kruskal

We said Kruskal is: $\mathcal{O}(|E| \log |E| + |E|(2 \times T_{find} + T_{union}))$

- $\mathcal{O}(|E|)$ unions and finds on $|V|$ objects is: $\mathcal{O}(|E| \alpha(|V|))$

Back to Kruskal

We said Kruskal is: $\mathcal{O}(|E| \log |E| + |E|(2 \times T_{find} + T_{union}))$

- $\mathcal{O}(|E|)$ unions and finds on $|V|$ objects is: $\mathcal{O}(|E| \alpha(|V|))$
- So we get: $\mathcal{O}(|E| \log |E| + |E| \alpha(|V|))$

Back to Kruskal

We said Kruskal is: $\mathcal{O}(|E| \log |E| + |E|(2 \times T_{find} + T_{union}))$

- $\mathcal{O}(|E|)$ unions and finds on $|V|$ objects is: $\mathcal{O}(|E| \alpha(|V|))$
- So we get: $\mathcal{O}(|E| \log |E| + |E| \alpha(|V|))$
- For almost* all graphs, left dominates: $\mathcal{O}(|E| \log |E|)$

Back to Kruskal

We said Kruskal is: $\mathcal{O}(|E| \log |E| + |E|(2 \times T_{find} + T_{union}))$

- $\mathcal{O}(|E|)$ unions and finds on $|V|$ objects is: $\mathcal{O}(|E| \alpha(|V|))$
- So we get: $\mathcal{O}(|E| \log |E| + |E| \alpha(|V|))$
- For almost* all graphs, left dominates: $\mathcal{O}(|E| \log |E|)$
- Which we saw is bounded by: $\mathcal{O}(|E| \log |V|)$

Back to Kruskal

We said Kruskal is: $\mathcal{O}(|E| \log |E| + |E|(2 \times T_{find} + T_{union}))$

- $\mathcal{O}(|E|)$ unions and finds on $|V|$ objects is: $\mathcal{O}(|E| \alpha(|V|))$
 - So we get: $\mathcal{O}(|E| \log |E| + |E| \alpha(|V|))$
 - For almost* all graphs, left dominates: $\mathcal{O}(|E| \log |E|)$
 - Which we saw is bounded by: $\mathcal{O}(|E| \log |V|)$
-
- * Except extremely sparse graphs, where $\alpha(|V|) > \log |E|$
 - ▶ Then we get $\mathcal{O}(|E| \alpha(|V|))$, which is $\subset \mathcal{O}(|E| \log |V|)$

Back to Kruskal

We said Kruskal is: $\mathcal{O}(|E| \log |E| + |E|(2 \times T_{find} + T_{union}))$

- $\mathcal{O}(|E|)$ unions and finds on $|V|$ objects is: $\mathcal{O}(|E| \alpha(|V|))$
- So we get: $\mathcal{O}(|E| \log |E| + |E| \alpha(|V|))$
- For almost* all graphs, left dominates: $\mathcal{O}(|E| \log |E|)$
- Which we saw is bounded by: $\mathcal{O}(|E| \log |V|)$

- * Except extremely sparse graphs, where $\alpha(|V|) > \log |E|$
 - ▶ Then we get $\mathcal{O}(|E| \alpha(|V|))$, which is $\subset \mathcal{O}(|E| \log |V|)$

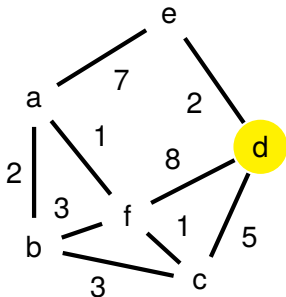
- **Bonus:** With some restrictions on edge weights, can get that down to $\mathcal{O}(|E| \alpha(|V|))$
 - ▶ I.e., the closest thing to $\mathcal{O}(|E|)$
 - ▶ Restriction is to use a *counting sort*, to sort in $\mathcal{O}(n)$
 - ▶ At which point we get: $\mathcal{O}(|E| + |E| \alpha(|V|)) = \mathcal{O}(|E| \alpha(|V|))$

Disjoint sets beyond Kruskal

- Disjoint sets are a pretty specialized ADT
 - ▶ Very specific roster of operations
- Still, comes in handy surprisingly often!
 - ▶ Detecting sets of duplicates, transitively
 - ▶ E.g., a bunch of image files that are all the same picture
 - ▶ Procedural generation
 - ▶ E.g., generating game levels that can be solved by players
 - ▶ Unification
 - ▶ Used in artificial intelligence (take Comp Sci 348!)
 - ▶ And to do type inference (take Comp Sci 321!)
 - ▶ Computing connected components of a graph
 - ▶ Used in parallelizing compilers (take Comp Sci 322!)
 - ▶ Etc.

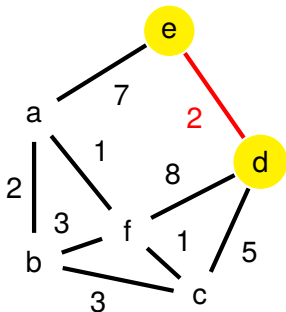
If we have time: Prim's algorithm

1. Start the tree at any vertex (they will all be in eventually, assuming we have a connected graph)
2. Find the smallest edge connecting a tree vertex to a non-tree vertex, and add it to the tree
3. Repeat until all vertices are in the tree



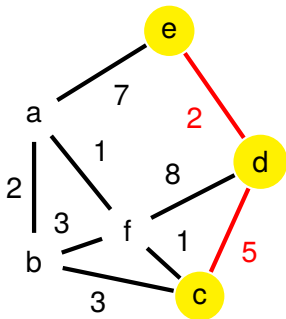
If we have time: Prim's algorithm

1. Start the tree at any vertex (they will all be in eventually, assuming we have a connected graph)
2. Find the smallest edge connecting a tree vertex to a non-tree vertex, and add it to the tree
3. Repeat until all vertices are in the tree



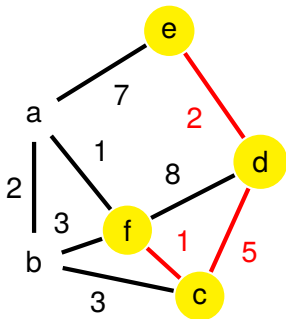
If we have time: Prim's algorithm

1. Start the tree at any vertex (they will all be in eventually, assuming we have a connected graph)
2. Find the smallest edge connecting a tree vertex to a non-tree vertex, and add it to the tree
3. Repeat until all vertices are in the tree



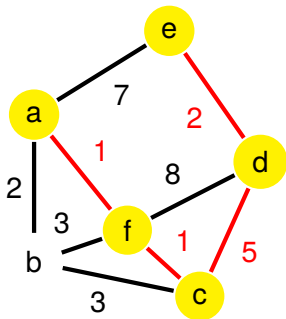
If we have time: Prim's algorithm

1. Start the tree at any vertex (they will all be in eventually, assuming we have a connected graph)
2. Find the smallest edge connecting a tree vertex to a non-tree vertex, and add it to the tree
3. Repeat until all vertices are in the tree



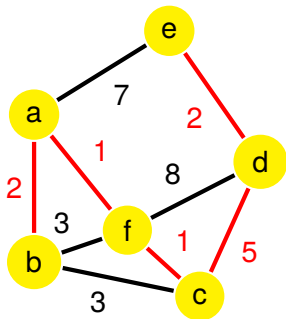
If we have time: Prim's algorithm

1. Start the tree at any vertex (they will all be in eventually, assuming we have a connected graph)
2. Find the smallest edge connecting a tree vertex to a non-tree vertex, and add it to the tree
3. Repeat until all vertices are in the tree



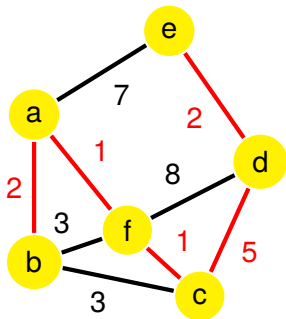
If we have time: Prim's algorithm

1. Start the tree at any vertex (they will all be in eventually, assuming we have a connected graph)
2. Find the smallest edge connecting a tree vertex to a non-tree vertex, and add it to the tree
3. Repeat until all vertices are in the tree



If we have time: Prim's algorithm

1. Start the tree at any vertex (they will all be in eventually, assuming we have a connected graph)
2. Find the smallest edge connecting a tree vertex to a non-tree vertex, and add it to the tree
3. Repeat until all vertices are in the tree



Conceptually simple, but lots of cases! Tricky to implement.
Can only find a single MST; not a full forest (unlike Kruskal).

More examples

- <https://www.cs.usfca.edu/~galles/visualization/Prim.html>
- <https://www.cs.usfca.edu/~galles/visualization/Kruskal.html>
- <https://www.cs.usfca.edu/~galles/visualization/DisjointSets.html>
 - ▶ His disjoint sets are slightly different from ours, but equivalent.

Next time: Binary Search
Trees